

O'REILLY®

Evals for AI Engineers

Systematically Measuring and Improving
AI Applications



Early
Release

RAW &
UNEDITED

Shreya Shankar
& Hamel Husain

Evals for AI Engineers

Systematically Measuring and Improving AI Applications

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

Shreya Shankar and Hamel Husain

Evals for AI Engineers

by Shreya Shankar and Hamel Husain

Copyright © 2027 Hamel Husain and Shreya Shankar. All rights reserved.

Published by O’Reilly Media, Inc., 141 Stony Circle, Suite 195, Santa Rosa, CA 95401.

O’Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<https://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: Nicole Butterfield

Development Editor: Angela Rufino

Production Editor: Katherine Tozer

Interior Designer: David Futato

Interior Illustrator: Kate Dullea

September 2026: First Edition

Revision History for the Early Release

- 2025-10-30: First Release
- 2025-11-13: Second Release
- 2026-01-26: Third Release
- 2026-03-06: Fourth Release

- 2026-04-30: Fifth Release
- 2026-05-28: Sixth Release

See <https://oreilly.com/catalog/errata.csp?isbn=9798341660724> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Evals for AI Engineers*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors and do not represent the publisher's views. While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

979-8-341-66067-0

Brief Table of Contents *(Not Yet Final)*

Chapter 1: Introduction (available)

Chapter 2: LLMs and Evaluation Basics (available)

Chapter 3: Error Analysis (available)

Chapter 4: Collaborative Evaluation Practices (available)

Chapter 5: Implementing Automated Evaluators (available)

Chapter 6: Evaluating Multi-Turn Conversations(available)

Chapter 7: Evaluating Retrieval-Augmented Generation(available)

Chapter 8: Evaluating Tool Use and Complex Agents (available)

Chapter 9: Continuous Integration and Deployment for LLM Agents (available)

Chapter 10: Interfaces for Human Review (available)

Chapter 11: Data Analysis for Traces (available)

Chapter 12: Improving LLM Agents (available)

Preface

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

If you’d like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at arufino@oreilly.com.

Large language models have moved from research curiosity to production reality with remarkable speed. Organizations now routinely embed LLMs in customer service systems, content generation pipelines, decision support tools, and information extraction workflows. Yet this rapid adoption has outpaced our ability to systematically evaluate whether these systems actually work.

This book addresses that gap.

Unlike traditional software with deterministic outputs, LLM pipelines produce responses that are often subjective, context-dependent, and multifaceted. A response might be factually accurate yet inappropriate for the context. It might sound persuasive while conveying incorrect information. It might also address most but not all parts of the user’s question. These ambiguities make evaluation fundamentally different from conventional software testing or even traditional machine learning validation.

The challenge we tackle is straightforward to state but difficult to solve: How do you assess whether an LLM pipeline is performing adequately? How do you diagnose where it’s failing? And how do you systematically improve it?

What This Book Covers

This book introduces a practical framework for evaluating LLM pipelines: the *Analyze-Measure-Improve* lifecycle. This iterative approach helps you:

- Analyze your pipeline’s behavior on representative data to identify failure modes
- Measure those failures quantitatively using automated evaluators
- Improve your system based on concrete evidence rather than guesswork

We cover the complete evaluation journey, from initial error analysis through production monitoring:

- Understanding the unique challenges of LLM evaluation (the “Three Gulfs” model)

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

- Systematic error analysis using qualitative coding techniques
- Collaborative evaluation practices for building shared understanding across teams
- Implementing automated evaluators, including LLM-as-judge approaches
- Evaluating specific architectures: multi-turn conversations, RAG systems, tool-calling, and agentic systems
- Continuous integration and deployment for LLM pipelines
- Building interfaces for ongoing human review and error analysis
- Strategies for improvement: accuracy optimization, cost reduction, and model cascades

Each chapter combines conceptual frameworks with practical implementation details. You'll find code examples, real-world case studies, common pitfalls to avoid, and exercises to test your understanding.

The material is based on a **live course** we teach for engineers and product managers who build production AI systems.

Who Should Read This Book

This book is written for engineers and technical product managers building software products with generative AI components. We assume you have:

- Familiarity with large language models and how they work
- Experience building (or planning to build) applications that use LLM APIs
- A practical need to know whether your LLM pipeline is actually working

You don't need to be an expert in machine learning or statistics. While we include code examples and some mathematical notation, we always explain the underlying concepts. If you're not comfortable with code or math, focus on the high-level frameworks and practical patterns, you'll still gain tremendous value from understanding the evaluation process, forming the right mental models, and recognizing common pitfalls.

Product managers will find particular value in the chapters on error analysis, collaborative evaluation, and the strategic framework for thinking about evaluation across the product lifecycle. Engineers will appreciate the detailed implementation guidance, code examples, and production deployment strategies.

What This Book Doesn't Cover

This book focuses exclusively on *application-centric evaluation*, or assessing whether your specific LLM pipeline works for your specific use case. We deliberately don't cover:

- General-purpose foundation model benchmarking (like MMLU, HellaSwag, or other academic benchmarks)
- Training or fine-tuning foundation models from scratch
- Comparative model selection based solely on benchmark performance

If you're trying to decide which foundation model to use, start by prototyping with your actual application and data, then use the techniques in this book to evaluate whether each model meets your requirements.

How This Book Is Organized

The book follows a logical progression through the evaluation lifecycle:

Part I: Foundations

(Chapters 1-2) establishes why LLM evaluation is uniquely challenging and introduces the core concepts you'll need.

Part II: The Analyze Phase

(Chapters 3-4) teaches systematic error analysis and collaborative evaluation practices for understanding what your pipeline is actually doing.

Part III: The Measure Phase

(Chapters 5-8) covers implementing automated evaluators and handling specific architectures like RAG, multi-turn conversations, and agentic systems.

Part IV: Production & Improvement

(Chapters 9-11) addresses continuous integration, monitoring, interfaces for ongoing review, and systematic improvement strategies.

You can read the book linearly or jump to chapters relevant to your immediate needs. However, we strongly recommend reading Chapters 1-3 before diving into later sections, as they establish the foundational framework that everything else builds upon.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

`Constant width`

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

Constant width bold

Shows commands or other text that should be typed literally by the user.

Constant width italic

Shows text that should be replaced with user-supplied values or by values determined by context.



TIP

This element signifies a tip or suggestion.

NOTE

This element signifies a general note.

ASPOSE
Your File Format APIs

WARNING

This element indicates a warning or caution.

Using Code Examples

Supplemental material (code examples, exercises, etc.) is available for download at <https://github.com/oreillymedia/ai-evals-book>.

If you have a technical question or a problem using the code examples, please send email to support@oreilly.com.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting

example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: "*Evaluating LLM Applications* by Shreya Shankar and Hamel Husain (O'Reilly). Copyright 2026 Shreya Shankar and Hamel Husain, 978-0-596-xxxx-x."

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

O'Reilly Online Learning

NOTE

For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <https://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.

1005 Gravenstein Highway North

Sebastopol, CA 95472

800-889-8969 (in the United States or Canada)

707-827-7019 (international or local)

707-829-0104 (fax)

support@oreilly.com

<https://www.oreilly.com/about/contact.html>

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at <https://learning.oreilly.com/library/view/evals-for-ai/9798341660717/>.

For news and information about our books and courses, visit <https://oreilly.com>.

Find us on LinkedIn: <https://linkedin.com/company/oreilly>

Watch us on YouTube: <https://youtube.com/oreillymedia>

Acknowledgments

This book would not exist without the engineers and product managers who have participated in our course at <https://evals.info>, and the companies we've partnered with through consulting engagements. Your real-world challenges, questions, feedback, and shared experiences have shaped every chapter.

We're grateful to our technical reviewers [NAMES TK] for their thorough feedback and for catching our mistakes.

Thanks to our editor [NAME TK] at O'Reilly for believing in this project and for patience as we translated course materials into book form.

A Note on the Pace of Change

The field of LLM evaluation is evolving rapidly. By the time you read this, new models, tools, and techniques will have emerged. However, the core principles in this book; understanding your data, systematically analyzing failures, measuring what matters, and improving based on evidence; remain constant regardless of which model or framework you're using.

We've designed the Analyze-Measure-Improve framework to be tool-agnostic and model-agnostic. If a new LLM comes out tomorrow, the methodology in this book will still help you evaluate whether it works for your use case.

Let's get started.

Chapter 1. Introduction

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 1st chapter of the final book. If you'd like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at arufino@oreilly.com.

The past several years have seen rapid advances in the development and deployment of large language models (LLMs). Organizations are embedding LLMs into software products for customer service, content creation, decision support, and information extraction, among many others ([Link to Come], Bommasani et al. 2021, [Link to Come], Zaharia et al. 2024). We call these software products *LLM applications*, and the LLM-driven component inside each one (the part that makes decisions, calls tools, and produces output) the *LLM agent*.

WHAT IS AN “AGENT”?

We use “agent” broadly throughout the book. It covers the full range from a single LLM call with no tools (the simplest case) to a system that runs in a loop, calls many tools, and makes many decisions per request. **Chapter 2** walks through this range with concrete examples.

The distinction between agent and application is useful for evaluation. Some failures come from the agent itself (wrong tool call, incorrect reasoning, bad output format). Others come from the application around it (broken UI, missing data, slow response times). This book focuses on agent-level evaluation.

Adoption is outpacing our ability to systematically evaluate these LLM applications ([Link to Come], Ward 2024). The consequences of shipping an unevaluated agent can be significant. In one illustrative story, in late 2022, a passenger named Jake Moffatt visited the Air Canada website to book a last-minute flight after his grandmother died. He asked the airline’s customer support chatbot whether he could get a *bereavement fare* (a discounted ticket that many airlines offer to people traveling for a funeral or death in the family). The chatbot incorrectly told him he could book the regular fare and apply for the bereavement discount as a refund within 90 days of the flight. Air Canada’s actual policy was that bereavement discounts could only be requested *before* the flight, not after. Later, when Moffatt applied for the refund, the airline refused. He took the case to the British Columbia Civil Resolution Tribunal, which ruled in his favor in February 2024 ([Link to Come], Moffatt v. Air Canada 2024). The tribunal rejected the airline’s argument, that the chatbot was a “separate legal entity,” and held Air Canada liable for the hallucinated policy. The airline was ordered to pay damages, and removed the chatbot from its website soon after.

No one knows exactly why the chatbot produced the wrong answer. It could have been trained or prompted on policy text that it then paraphrased incorrectly. It could have incorrectly referenced language from other airlines’ policies that do allow retroactive bereavement claims. What we do know is that nobody caught the failure before a customer relied on it. This is the kind of issue that evaluation is designed to surface: a systematic check of the chatbot’s answers on common policy questions could have flagged the discrepancy before the chatbot went live. Cases like this illustrate why evaluation is necessary. Agents that give wrong answers in production can lead to serious legal and reputational costs.

Unlike traditional software, LLM applications do not produce deterministic responses, and they have no formal specifications for the logic they should and should not perform. Their outputs are unpredictable. Sometimes, as in the airline story, they are obviously incorrect. Other times, the issue is harder to describe. An output can be factually right but inappropriate (the “vibes are off”), or sound convincing while being totally wrong. These ambiguities make evaluation fundamentally different from conventional software testing and even from traditional machine learning (ML).¹

The core challenge of evaluation is as follows: How do we assess whether an LLM pipeline is performing adequately? And how do we diagnose where it is failing?

Our book focuses on this gap. In this book, we will build a practical and theoretical foundation for LLM evaluation, with methods that you can immediately apply to your own projects.

What is Evaluation?

Evaluation is the *systematic measurement of quality in an LLM application*. A good evaluation produces results that are interpretable and actionable. In some cases evaluation produces a quantitative metric. For example, if you are building a travel planning assistant, you might measure how often the system returns valid flight and hotel options within budget, or whether itineraries satisfy the user’s constraints. In other cases, the “metric” is generated with more judgment involved. A set of experts, or sometimes the product team itself, may review outputs and rate them for qualities such as clarity, tone, or usefulness. Both approaches fall on a spectrum of rigor: from quick, informal checks that guide early iteration, to carefully defined rubrics and systematic sampling that support production systems.

A single application will usually require multiple evaluations, each designed to capture a different dimension of performance. Evaluations can then be integrated into the application in several ways:

Monitoring

continuous checks that run in the background to detect drift or degradation over time.

Guardrails

evaluations placed in the critical path, allowing the system to block, retry, or fall back when outputs fail predefined criteria.

Improvement mechanisms

evaluations used to generate feedback, such as labeled examples for fine-tuning, curated few-shot prompts, or systematic identification of failure cases that motivate architectural changes.

So, overall, evaluations serve multiple purposes. They are necessary to detect errors, maintain user trust, enforce safety constraints, monitor ongoing behavior, and guide systematic improvement.

Without them, teams are left uncertain about how their systems perform and where they need to intervene.

Evaluation Throughout the Lifecycle of an LLM

Evaluation is not confined to applications of LLMs. It is a constant presence across the entire lifecycle of an LLM. Interestingly, at each stage (pre-training, post-training, and then any applications we build to top) the role of evaluation is different, reflecting distinct goals and constraints. Understanding these stages helps clarify the spectrum of capabilities being tested, and why there can be no single evaluation framework that applies everywhere.

Pre-training

Pre-training is the first stage in the lifecycle, where a foundation model is built from scratch. A large neural network is exposed to massive text (and sometimes code) collections with a single objective, predicting the next *token* in a sequence. A token is the smallest unit of text the model reads or writes at a time. It is usually a short sequence of characters, roughly corresponding to a word or part of a word. For example, the sentence “The chatbot answered the question” might be broken up into tokens like The, chat, bot, answered, the, question. Models read and generate one token at a time, and the cost of an API call is usually measured in the number of tokens in and out. During pretraining, the model learns broad linguistic knowledge and some capacity for reasoning and world knowledge by doing this next-token prediction over billions of examples.

Pre-training is carried out by large research labs and commercial model providers. The process requires substantial computational resources, often costing hundreds of millions of dollars. Evaluation at this stage is largely intrinsic: measuring how accurately the model predicts held-out tokens from its training distribution. Pretraining metrics indicate whether the model has absorbed the statistical patterns of language, but do not yet assess its usefulness for many of the downstream tasks that users want to leverage LLMs for.

Post-training

Post-training takes a pre-trained model and adapts it for practical use. It turns out that models trained only on next-token prediction can already generalize to many unseen tasks, or tasks that LLMs weren’t explicitly pre-trained on. Post-training builds on this ability by steering the model toward behaviors that users care about; e.g., following instructions, staying on topic, and avoiding unsafe responses.

Post-training usually has two parts. First, the model provider fine-tunes the model on curated examples of prompts paired with good responses, so the model learns to produce answers in the format people want. Second, they use preference data (pairs of responses where a human picked which one was better) to further nudge the model toward the preferred style. We do not cover these techniques in detail in this book because most engineers interact with the model after these steps are done. Evaluation at

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

this stage shifts from measuring raw next-token accuracy to running the model on benchmarks that approximate user-facing tasks like question-answering, reasoning challenges, safety and bias tests, and coding. These evaluations are designed and run by the model providers themselves, and their results are often reported as headline benchmarks.

Application

The application stage is where most engineers and product managers engage with LLMs. At this point the model has already been trained and aligned by a model provider, and the task is to integrate it into a concrete workflow, e.g., triaging emails, assisting customer support, or analyzing documents.

Evaluation at this stage must be tailored to the goals and data of the specific application. In some cases, existing post-training evaluations overlap directly with the task. For example, a coding assistant can draw on established programming benchmarks. In most cases, however, the application is not represented in public leaderboards. A system for reviewing contracts, summarizing medical notes, or checking compliance will require its own evaluation methods.

Responsibility therefore falls to the application development team to define what success means, design appropriate metrics, and implement monitoring to verify that the system behaves reliably for its intended use.

Evaluation in Applications

At the application stage, the model has already been trained. The work is in building around it. You have to decide what the product should achieve, translate that goal into prompts and agent logic, and then verify that the model's outputs are acceptable. Each of these steps introduces its own difficulties:

Objectives are vague at first

A goal like “summarize emails” sounds simple, but only when you inspect outputs do the ambiguities appear: how detailed, in what format, and with what exclusions?

Instructions might have gaps

Prompts and agent logic expressed in natural language rarely capture the full intent, which leads to inconsistent or incomplete behavior.

Behavior changes on new data

A setup that works on a handful of test cases may fail once it encounters the variability of real inputs.

In practice, these issues show up again and again when building LLM applications. To reason about these challenges more systematically, we use the *Three Gulfs* framework, which names three places where development tends to break down — comprehension, specification, and generalization. Each

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

gulf highlights a different point where development typically goes wrong, and each requires a different strategy to address, a theme we will return to throughout this book.

The Three Gulfs of LLM Application Development

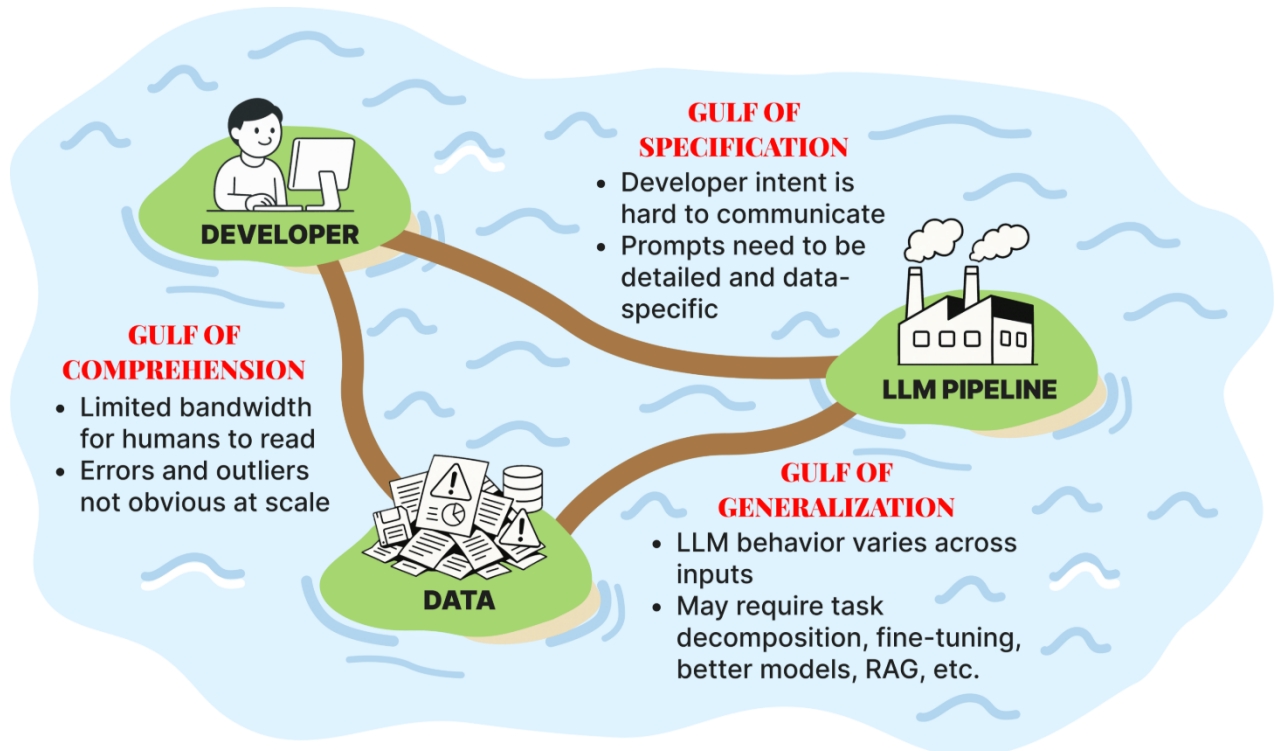


Figure 1-1. Three Gulfs Model of Challenges in LLM Pipeline Development. Adapted from *Steering Semantic Data Processing with DocWrangler* Shankar 2025.

A useful way to understand the challenges of building with LLMs is the Three Gulfs model (Figure 1-1), adapted from ([Link to Come], Shankar et al. 2025) and inspired by ([Link to Come], Norman 1988). The model highlights three common gaps that developers face when building and evaluating applications. We call these gaps comprehension, specification, and generalization. Later in this book, we will use this framework to diagnose why an application is failing, and select the most appropriate strategy to fix it.

EXAMPLE: EMAIL PROCESSING PIPELINE

Consider a pipeline that processes incoming emails sent to an organization. The goal is to extract the sender's name, summarize the key requests, and categorize the email. At first glance, this seems straightforward. But each stage of development exposes different challenges across the Three Gulfs.

The Gulf of Comprehension (Between the Developer and Data)

The first gulf is between the developer and the data. To build effective prompts or pipelines, you need a solid grasp of what your inputs actually look like. Without that, you will design against an imagined version of the task, not the real one.

Consider the email processing example. If you only look at a handful of clean, well-written emails when drafting your prompt, you might write instructions that work fine on those cases. But the production data will also include forwards, replies with long quoted threads, messages with signatures in multiple languages, and poorly formatted text copied from PDFs. A prompt that ignores these realities might lead the LLM to, for example, summarize an entire quoted thread instead of the new message, or mistake a signature block for the main request.

The gulf of comprehension applies to outputs too, not just inputs. We also need a clear sense of how the LLM behaves on those inputs. You cannot read every output it generates, but you still need to recognize the common ways it succeeds and the common ways it fails. Otherwise, it's easy to be misled by a few good examples and miss the patterns that only show up at scale. In the email agent, for example, the LLM might usually extract the sender and write a decent summary. But when you look closer, you may notice recurring mistakes. The LLM might pull the wrong name when several are mentioned, drop key details in longer messages, or copy boilerplate like confidentiality notices into the summary. These failures may only happen occasionally, but if they occur, the system will feel unreliable to users.

Bridging the Gulf of Comprehension means developing strategies to characterize both your inputs and your outputs at scale—so you know what the data really looks like, and how the system actually behaves—without having to inspect everything by hand.

The Gulf of Specification (Between the Developer and the LLM Pipeline)

The second gulf is between what we want and what we actually tell the LLM to do. Our intent—the task we have in mind—is often only loosely captured by the prompts or pipeline logic we write. Natural language is imprecise, and small gaps in how we phrase things can lead to large differences in behavior.

Take the instruction:

Extract the sender's name and summarize the key requests in this email.

At first glance it looks specific, but there are important details left unstated. Should the summary be a paragraph or a bulleted list? Should the sender be the display name, the full email address, or both? Should implicit requests be included, or only explicit ones? How concise should the summary be?

The LLM cannot fill in these choices for you. If you don't make them explicit, the system will guess—and those guesses will not always line up with what you expect. That's why two runs of the same

pipeline might produce outputs in different formats, with varying levels of detail, or with inconsistent handling of edge cases.

The Gulf of Specification is this gap between our intent and our instructions. Closing it requires making expectations explicit and aligning them closely with the data the pipeline will actually see.

The Gulf of Generalization (Between the Data and the LLM Pipeline)

This document was truncated here because it was created in the Evaluation Mode.

